
Projet Breezy

Réseau social fullstack moderne : architecture microservices, web & mobile

Rapport de présentation et de réalisation d'un projet logiciel



CESI 
ÉCOLE D'INGÉNIEURS

ÉQUIPE PROJET

Antonin RABATEL · Mathéo CARVAL · Youcef AFANE · Enzo CADIÈRE

Année universitaire 2025 – 2026



Introduction	5
Table des figures	40
1. Présentation générale de Breezy	6
1.1 Vision et concept.....	6
1.2 Problématique adressée	6
1.3 Objectifs fonctionnels et techniques	6
1.4 Public cible	6
1.5 Valeur ajoutée et différenciation.....	6
1.6 Positionnement vis-à-vis de Twitter / X.....	7
2. Vue d'ensemble de la plateforme.....	8
2.1 Fonctionnement global (web + mobile).....	8
2.2 Parcours utilisateur type	8
2.3 Logique métier centrale	9
2.4 Rôle des modules dans l'écosystème	9
3. Architecture globale.....	10
3.1 Architecture logicielle globale (monorepo)	10
3.2 Architecture des microservices et flux de communication.....	10
3.3 Architecture frontend (SPA React / Vite).....	10
3.4 Architecture mobile (Capacitor)	11
3.5 Architecture base de données (persistance polyglotte)	12
3.6 Architecture API (gateway, routage, authentification).....	13
3.7 Architecture temps réel (SSE + PostgreSQL LISTEN/NOTIFY).....	13
3.8 Architecture de stockage (MinIO, médias, chiffrement)	13
4. Choix technologiques détaillés	14
4.1 Backend.....	14
4.2 Frontend.....	14
4.3 Mobile, infrastructure et qualité.....	15
4.4 Server-Sent Events plutôt que WebSocket	15
5. Structure du projet.....	16
5.1 Organisation du monorepo et workspaces NPM	16
5.2 Séparation des responsabilités	16
5.3 Conventions et patterns architecturaux	16
6. Fonctionnalités du réseau social.....	17
6.1 Authentification complète	17
6.2 Profils utilisateurs.....	17
6.3 Publication : posts, réponses, citations, médias, sondages	18



6.4 Interactions : likes, rebreeze, signets.....	19
6.5 Hashtags, tendances et recherche.....	19
6.6 Suivi, comptes privés et blocage.....	19
6.7 Feed personnalisé « For You ».....	19
6.8 Messagerie directe chiffrée.....	19
6.9 Notifications temps réel.....	20
6.10 Modération et administration.....	21
7. Expérience utilisateur.....	22
7.1 Parcours et navigation.....	22
7.2 Design system.....	22
7.3 Thème, responsive et internationalisation.....	22
7.4 Accessibilité.....	22
8. Frontend Web.....	23
8.1 Composants et pages.....	23
8.2 Routage et protection des routes.....	23
8.3 Gestion d'état hybride.....	23
8.4 Hooks personnalisés et accès API.....	23
8.5 Gestion de l'upload de médias.....	23
8.6 Optimisation des performances et configuration Vite.....	24
8.7 Paquet partagé @breezy/shared.....	24
9. Backend — Micro-services Go.....	25
9.1 API Gateway.....	25
9.2 Services et responsabilités.....	25
9.3 Communication inter-services.....	25
9.4 Middlewares, validation et rôles.....	25
10. Base de données.....	26
10.1 Architecture polyglotte.....	26
10.2 Modèles structurants.....	26
10.3 Relations et schéma.....	26
10.4 Choix de PostgreSQL.....	26
11. Temps réel.....	27
11.1 Flux de notification de bout en bout.....	27
11.2 Types d'événements et justification du choix.....	27
12. Sécurité.....	28
12.1 Authentification JWT et refresh tokens.....	28
12.2 MFA, OAuth2 et vérifications.....	29



12.3 Chiffrement, limitation de débit et durcissement	30
13. Performances	31
14. Déploiement et infrastructure	32
14.1 Environnements et développement local	32
14.2 Pipeline CI/CD (GitHub Actions)	32
14.3 Kubernetes, GitOps et TLS	32
14.4 Stratégie d'images et monitoring	33
15. Gestion de projet	34
15.1 Méthodologie GitHub-centric	34
15.2 Stratégie de branches et bénéfices	34
15.3 Documentation et répartition	34
16. Application mobile	36
16.1 Architecture Capacitor et partage de code	36
16.2 Spécificités natives et builds	36
17. Difficultés techniques rencontrées	37
18. Évolutions futures envisageables	38
18.1 Fonctionnelles	38
18.2 Techniques	38
19. Conclusion	39
Glossaire	41



Introduction

Breezy est un réseau social de type micro-blogging, comparable à Twitter / X, conçu pour rester léger et réactif tout en offrant un éventail complet de fonctionnalités sociales. Le service permet de publier des messages courts, de suivre d'autres utilisateurs, de réagir aux publications (likes, rebreeze, commentaires, signets), d'échanger en messagerie privée chiffrée, de recevoir des notifications en temps réel et de consulter un fil d'actualité personnalisé. Il est accessible à la fois par une application web et par une application mobile native (iOS et Android).

Au-delà du produit, ce projet constitue un exercice d'ingénierie mené dans le cadre du module Développement d'Applications Distribuées. Il vise à mettre en pratique la conception d'une architecture distribuée sept micro-services Go conteneurisés, persistance polyglotte, communication inter-services ainsi que l'industrialisation complète du développement : monorepo, intégration et déploiement continus, tests automatisés, orchestration Kubernetes et démarche GitOps.

Le présent rapport documente la réalisation du projet à destination d'un jury académique, d'un commanditaire potentiel et de recruteurs techniques. Il présente successivement la vision et le concept, l'architecture globale, les choix technologiques argumentés, la structure du projet, le détail des fonctionnalités, l'expérience utilisateur, les volets frontend et backend, la base de données, le temps réel, la sécurité, les performances, le déploiement, la gestion de projet, l'application mobile, les difficultés rencontrées et les évolutions envisagées.

1. Présentation générale de Breezy

1.1 Vision et concept

Breezy répond à une intention simple : offrir un espace de micro-blogging rapide, fluide et moderne, où la conversation publique et l'interaction sociale priment. Le concept reprend les codes éprouvés du format « fil court », messages concis, abonnements asymétriques (on suit sans réciprocité), interactions légères et les complète d'un socle technique contemporain : feed personnalisé, messagerie chiffrée, notifications temps réel et expérience identique sur le web et le mobile.

La vision technique est tout aussi structurante que la vision produit : démontrer qu'une équipe réduite peut concevoir, industrialiser et déployer une plateforme distribuée complète en s'appuyant sur des pratiques d'ingénierie professionnelles (microservices, GitOps, tests d'intégration réels, conteneurs durcis).

1.2 Problématique adressée

Un réseau social moderne doit concilier des exigences : réactivité perçue, richesse fonctionnelle, sécurité des échanges et capacité à montée en charge. Breezy traite cette tension en découplant le système en services autonomes, en mettant en cache les lectures fréquentes, en protégeant les accès par limitation de débit et en privilégiant des mécanismes temps réel légers (Server-Sent Events) plutôt que des protocoles plus lourds lorsque l'usage ne le justifie pas.

1.3 Objectifs fonctionnels et techniques

- **Objectif produit** : offrir une expérience de micro-blogging intuitive et mobile-first : publication, abonnements, interactions, feed personnalisé, messagerie et notifications.
- **Objectif d'ingénierie** : concevoir et déployer une architecture distribuée en sept micro-services conteneurisés, testée automatiquement et orchestrée via Kubernetes et Argo CD.
- **Objectif de sécurité** : garantir des échanges sûrs : JWT à rotation, MFA (TOTP), OAuth2, vérification d'e-mail, chiffrement des messages, limitation des abus.
- **Objectif de performance** : maintenir une application réactive grâce au cache Valkey, à la mémoïsation automatique (React Compiler) et à la gestion fine de l'état serveur.
- **Objectif de portabilité** : conteneuriser l'ensemble (images distroless) pour un déploiement reproductible du poste de développement à la production.

1.4 Public cible

Breezy s'adresse à un public généraliste familier des réseaux sociaux courts, recherchant une plateforme légère, réactive et respectueuse de la confidentialité (comptes privés, messages chiffrés, blocage). Pour le jury et les recruteurs, il constitue également une vitrine technique illustrant la maîtrise d'un écosystème distribué complet.

1.5 Valeur ajoutée et différenciation

- **Web et mobile unifiés** : une base de code unique sert le web et les applications natives via Capacitor, garantissant la cohérence fonctionnelle et réduisant le coût de maintenance.

- **Sécurité de premier plan** : MFA, OAuth2, rotation des refresh tokens par familles, chiffrement AES-256-GCM des messages et conteneurs durcis dépassent le niveau habituel d'un projet académique.
- **Temps réel sobre** : les notifications s'appuient sur SSE et le mécanisme LISTEN/NOTIFY de PostgreSQL, sans broker dédié, illustrant un choix d'architecture proportionné au besoin.
- **Industrialisation complète** : GitOps (Argo CD), CI/CD bloquante et tests d'intégration sur bases réelles (testcontainers) ancrent le projet dans des pratiques professionnelles.

1.6 Positionnement vis-à-vis de Twitter / X

Breezy reprend le modèle interactionnel de Twitter / X (fil court, abonnements asymétriques, likes, republication, ici nommée « rebreeze » —, mentions, hashtags) tout en assumant des choix propres : un feed « For You » construit par score thématique, une messagerie chiffrée au repos, et une architecture ouverte et auto-hébergeable (MinIO, Valkey, PostgreSQL) sans dépendance à un fournisseur propriétaire. L'objectif n'est pas de rivaliser à l'échelle d'un réseau mondial, mais de reproduire fidèlement ses mécanismes clés dans une architecture maîtrisable.

Partie prenante	Rôle	Attentes principales
Encadrant CESI / commanditaire	Maîtrise d'ouvrage pédagogique	Conformité à l'énoncé, qualité technique, respect de la méthodologie et des normes de rendu
Équipe projet (4 étudiants)	Maîtrise d'œuvre	Répartition claire des tâches, intégration continue, livrable fonctionnel et documenté
Utilisateurs finaux	Usage de l'application	Interface réactive, fluide, mobile-first ; échanges sécurisés et confidentialité maîtrisée
Mainteneurs (post-projet)	Maintenance / évolution	Code lisible, testé, documenté et facilement déployable (Docker, GitOps)

Tableau 1 — Parties prenantes et attentes

2. Vue d'ensemble de la plateforme

2.1 Fonctionnement global (web + mobile)

Breezy se présente sous deux formes partageant le même cœur applicatif : une application web monopage (SPA React / Vite) et des applications mobiles iOS / Android encapsulées par Capacitor. Les deux clients consomment une même API REST exposée par une API Gateway unique, qui route les requêtes vers les micro-services spécialisés. Les notifications sont reçues en direct via une connexion SSE persistante.

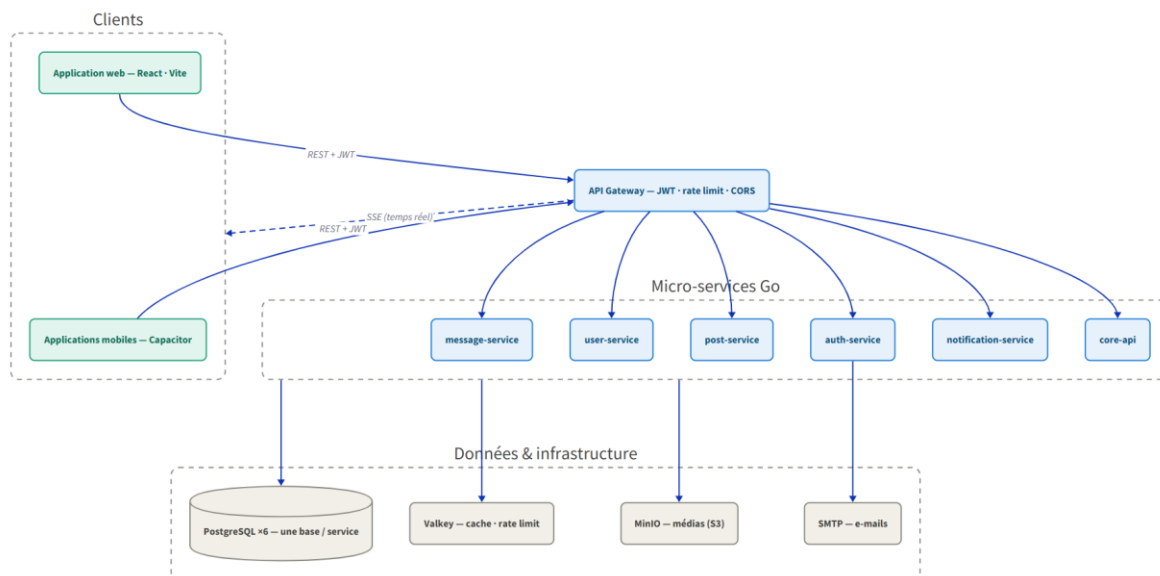


Figure 1 — Architecture globale de l'écosystème Breezy

2.2 Parcours utilisateur type

Le parcours nominal illustre l'enchaînement des modules. L'utilisateur s'inscrit, valide son adresse e-mail, se connecte (éventuellement renforcé par MFA), complète son profil, puis découvre le feed « For You ». Il suit des comptes, publie des messages, réagit (like, rebreeze, signet), commente, échange en messagerie privée et reçoit en direct des notifications.

- **Inscription & vérification** : création du compte (statut non vérifié), e-mail de validation, activation au clic.
- **Connexion** : authentification, émission d'un JWT court et d'un refresh token ; étape TOTP si la MFA est activée.
- **Découverte & interaction** : feed personnalisé, abonnements, publication, interactions, hashtags et recherche.
- **Échanges & temps réel** : messagerie chiffrée 1-à-1, notifications SSE (likes, follows, mentions, messages).

2.3 Logique métier centrale

La logique métier se répartit selon le domaine : l'identité et les sessions relèvent de l'auth-service ; le graphe social (profils, follows, blocages) du user-service ; les contenus et leurs interactions (posts, replies, citations, likes, rebreeze, signets, sondages, hashtags, signalements) du post-service ; les conversations du message-service ; et la diffusion d'événements du notification-service. Ce découpage par domaine métier (et non par couche technique) est au cœur de l'approche microservices.

2.4 Rôle des modules dans l'écosystème

Module	Rôle dans l'écosystème
API Gateway	Point d'entrée unique : validation JWT, rate limiting, CORS/CSRF, reverse proxy.
auth-service	Identité, sessions, MFA, OAuth2, e-mails transactionnels.
user-service	Profils, relations (follow/blocage), suggestions, modération.
post-service	Cœur social : posts, interactions, médias, hashtags, feed « For You ».
message-service	Messagerie directe chiffrée.
notification-service	Notifications temps réel (SSE + LISTEN/NOTIFY).
core-api	Service d'extension (base minimale dans la version actuelle).

Tableau 2 — Rôle des modules de la plateforme

3. Architecture globale

Cette section détaille l'architecture du système sous ses différentes facettes : organisation logicielle, microservices, frontend, mobile, base de données, API, temps réel et stockage. Conformément au format retenu, chaque diagramme est décrit par une spécification détaillée destinée à sa génération.

3.1 Architecture logicielle globale (monorepo)

L'ensemble du code est regroupé dans un monorepo géré par NPM Workspaces. Les applications (web, mobile, API) et les paquets partagés y cohabitent, ce qui simplifie le partage de configuration, la cohérence des versions et l'exécution d'une chaîne d'intégration unifiée. Le démarrage en développement lance automatiquement les bases PostgreSQL via Docker avant les serveurs.

3.2 Architecture des microservices et flux de communication

Le backend adopte une architecture microservices où chaque service possède une responsabilité métier claire et sa propre base de données. Les clients ne communiquent jamais directement avec un service interne : tout transite par l'API Gateway, qui assure les préoccupations transverses. Les services dialoguent entre eux en HTTP interne, authentifié par une clé d'API partagée (INTERNAL_API_KEY), selon un modèle « fire-and-forget » pour les traitements asynchrones (notamment les notifications). Le projet assume volontairement l'absence de broker (ni Kafka ni RabbitMQ), au profit de la simplicité.

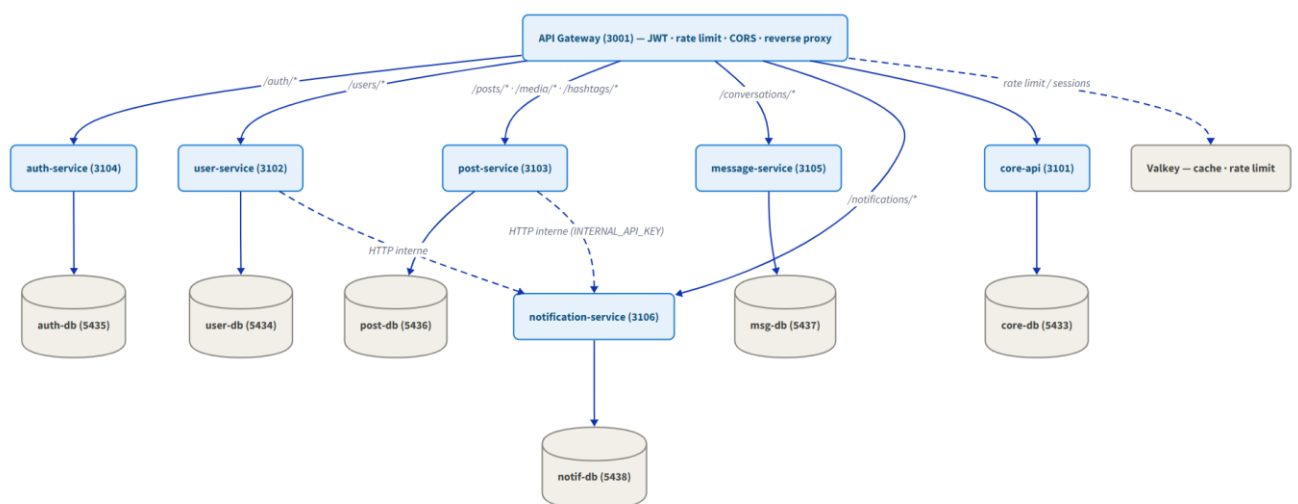


Figure 2 — Architecture des microservices et flux de communication

3.3 Architecture frontend (SPA React / Vite)

Le frontend est une application monopage structurée en couches, du routage jusqu'à l'accès réseau. Le routage (React Router 7) organise la navigation et la protection des routes ; les composants stylisés (TailwindCSS, shadcn/ui, Base UI) composent l'interface ; TanStack Query gère l'état serveur (cache,

invalidation, pagination) tandis que React Context porte l'état client (auth, thème, langue) ; enfin un client Axios centralise les appels HTTP et l'injection du JWT.

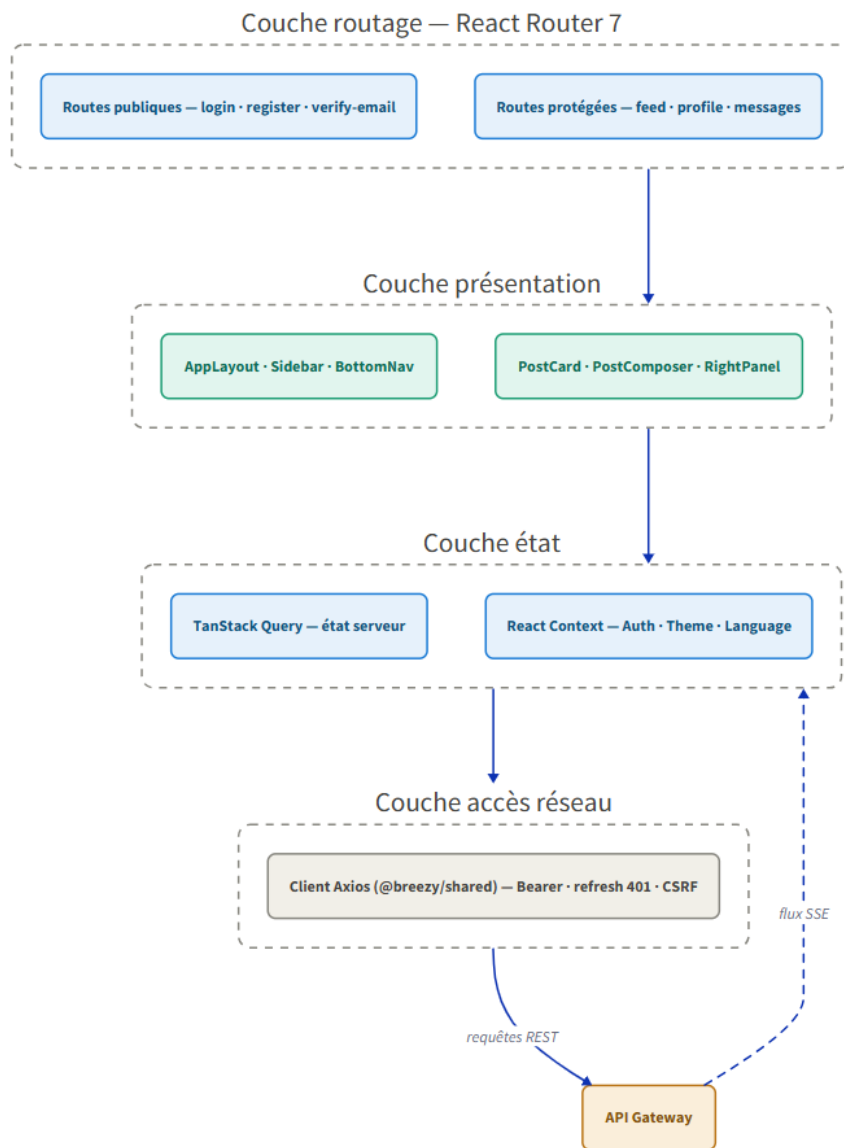


Figure 3 — Architecture en couches du frontend

3.4 Architecture mobile (Capacitor)

Les applications mobiles réutilisent intégralement le code du web : le même bundle, compilé en mode mobile, est exécuté dans une WebView native encapsulée par Capacitor 6. Le pont (bridge) Capacitor expose les capacités natives (caméra, notifications push, stockage chiffré, haptique, réseau) sous forme d'API JavaScript. Le stockage des jetons utilise l'API Preferences (chiffrée nativement) en lieu et place du localStorage du web.

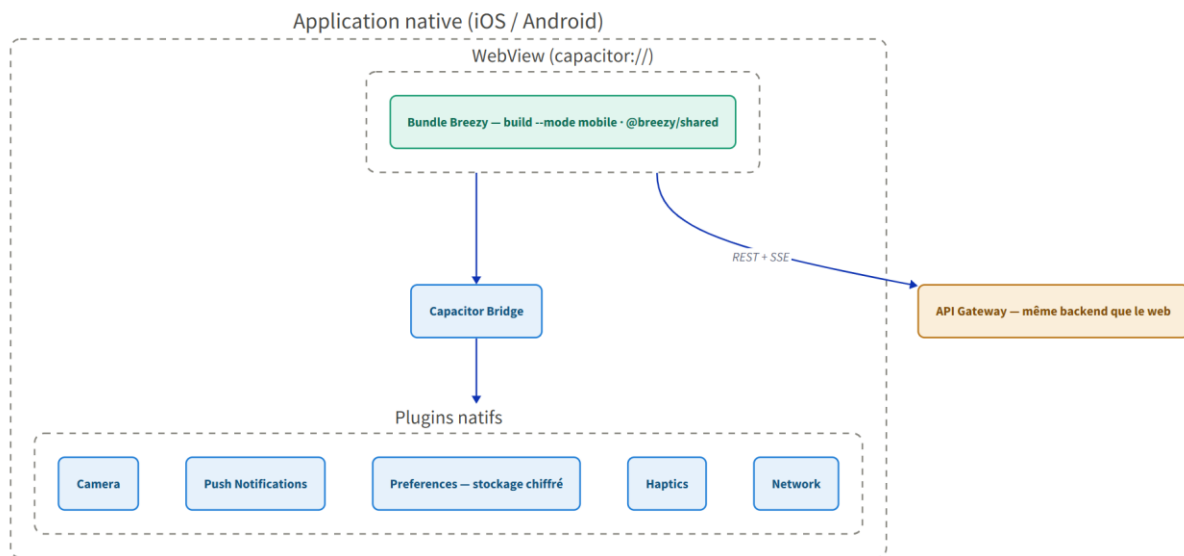


Figure 4 — Architecture mobile Capacitor (pont natif)

3.5 Architecture base de données (persistance polyglotte)

Breezy applique le pattern « database-per-service » : chaque micro-service possède sa propre base PostgreSQL isolée, ce qui garantit le découplage et l'autonomie d'évolution. Les références inter-services (par exemple l'identifiant d'auteur d'un post, géré par le user-service) sont des identifiants logiques sans contrainte de clé étrangère physique entre bases ; la cohérence est assurée par la logique applicative.

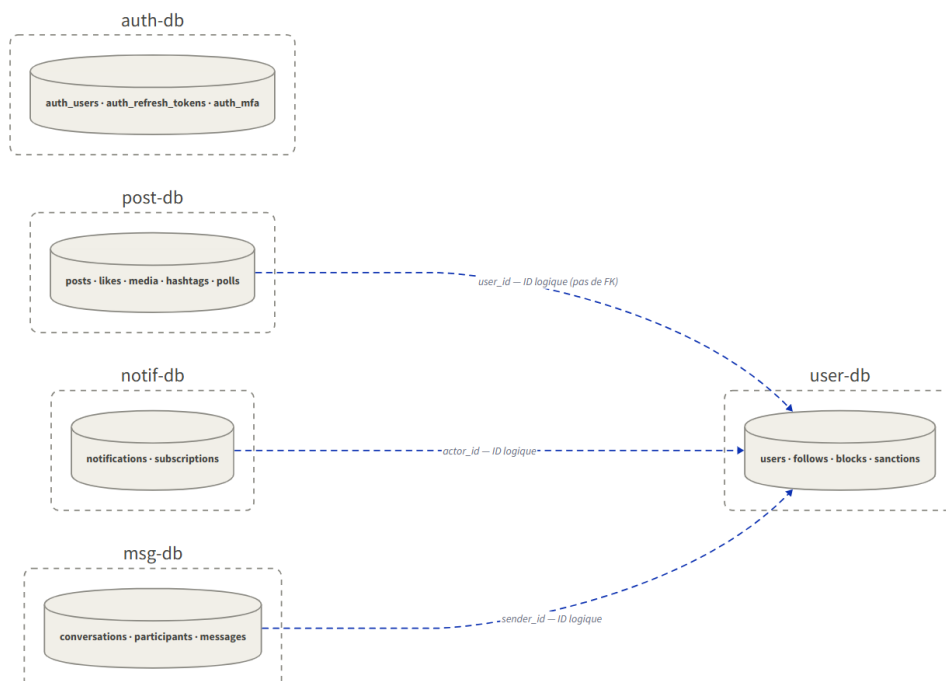


Figure 5 — Persistance polyglotte (une base par service)

3.6 Architecture API (gateway, routage, authentification)

L'API Gateway constitue la façade unique du système. Il valide les jetons JWT, applique la limitation de débit (par IP sur les routes d'authentification, par utilisateur sur l'API générale), vérifie la révocation de session via Valkey, impose la liste blanche CORS et la protection CSRF (en-tête X-Requested-With), puis relaie la requête vers le service concerné. Cette centralisation évite de dupliquer ces préoccupations dans chaque service.

3.7 Architecture temps réel (SSE + PostgreSQL LISTEN/NOTIFY)

Le temps réel repose sur les Server-Sent Events : le client ouvre une connexion HTTP persistante et reçoit les événements poussés par le serveur. Côté backend, le mécanisme LISTEN/NOTIFY de PostgreSQL sert de bus d'événements interne : la persistance d'une notification déclenche un signal que le gestionnaire SSE relaie immédiatement au client. Le flux détaillé est présenté en section 11.

3.8 Architecture de stockage (MinIO, médias, chiffrement)

Les binaires (avatars, bannières, médias des posts et des messages) sont stockés dans MinIO, un service de stockage objet compatible S3 et auto-hébergé. Les fichiers, jusqu'à 300 Mo, sont téléversés via un endpoint dédié puis référencés par leur clé d'objet. Les messages privés sont, eux, chiffrés au repos en AES-256-GCM.

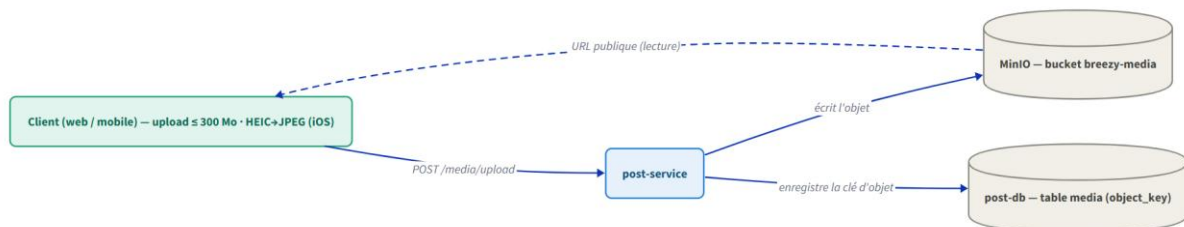


Figure 6 — Stockage et service des médias (MinIO)

4. Choix technologiques détaillés

Les technologies retenues répondent à trois critères : adéquation au besoin, robustesse éprouvée et cohérence d'ensemble. Cette section justifie chaque choix et en discute les compromis.

4.1 Backend

Technologie	Rôle	Avantages	Justification du choix
Go 1.25+	Langage backend	Compilation native, concurrence (goroutines), typage statique, binaires légers	Performances et faible empreinte mémoire, idéal pour des microservices conteneurisés
Gin	Framework HTTP	Léger, routeur rapide, chaîne de middlewares	Mise en place expressive des middlewares (JWT, CORS, rate limit, logs)
GORM	ORM	Productivité, migrations, repository pattern	Accès aux données lisible ; compromis de perf accepté face au gain de vélocité
PostgreSQL 17	Base de données	ACID, robustesse, LISTEN/NOTIFY	Transactions fiables + bus d'événements natif pour le temps réel
Valkey	Cache / rate limit	Compatible Redis, léger, open source	Limitation de débit distribuée et révocation de sessions, sans verrou propriétaire
MinIO	Stockage objet	Compatible S3, auto-hébergé	Pas de vendor lock-in ; gestion scalable des médias volumineux

Tableau 3 — Choix technologiques backend

Le couple Go + Gin offre un excellent rapport performance / lisibilité pour des services réseau. GORM accélère le développement au prix d'un léger surcoût à l'exécution, jugé acceptable à l'échelle du projet. PostgreSQL apporte la double valeur de garanties transactionnelles fortes et d'un mécanisme de notification interne réutilisé pour le temps réel.

4.2 Frontend

Technologie	Rôle	Avantages / justification
React 19	Bibliothèque UI	Écosystème mature ; React Compiler pour la mémoïsation automatique des composants
Vite	Outil de build	Démarrage et HMR très rapides ; modes de build web et mobile
TypeScript	Langage	Typage statique fullstack, sécurité et confort de développement
TanStack Query	État serveur	Cache, déduplication, invalidation, pagination (infinite scroll)
React Router 7	Routage	Navigation déclarative et protection des routes
TailwindCSS 4	Style	Utility-first, cohérence du design system, bundle optimisé
shadcn/ui + Base UI	Composants	Composants accessibles et personnalisables

Tableau 4 — Choix technologiques frontend

4.3 Mobile, infrastructure et qualité

Technologie	Rôle	Avantages / justification
Capacitor 6	Pont mobile	Partage du codebase web ; accès natif ; alternative légère à React Native
Docker + Distroless	Conteneurisation	Reproductibilité ; images sans shell, non-root → surface d'attaque minimale
Kubernetes (K3s)	Orchestration	K3s léger avec Traefik intégré ; scaling horizontal des services sans état
Argo CD	GitOps	Déploiement déclaratif, synchronisé depuis le dépôt, auditable
Kustomize	Config multi-env	Overlays dev/prod sans duplication de manifests
Sealed Secrets	Secrets	Secrets chiffrés versionnés dans le dépôt Git
Cert-Manager + Let's Encrypt	TLS	Certificats automatisés (challenge DNS-01 via Route53)
GitHub Actions	CI/CD	Lint, tests, build, publication des images vers GHCR
testcontainers-go	Tests d'intégration	Bases PostgreSQL réelles éphémères → tests fidèles

Tableau 5 — Choix technologiques mobile, infrastructure, qualité

4.4 Server-Sent Events plutôt que WebSocket

Pour les notifications, le besoin est unidirectionnel : le serveur pousse des événements vers le client, sans flux montant temps réel. Les Server-Sent Events répondent exactement à ce besoin, sur HTTP/1.1 standard, avec reconnexion automatique native et une mise en œuvre nettement plus simple qu'un WebSocket bidirectionnel. Le choix de SSE est donc proportionné : il évite la complexité d'un canal full-duplex inutile ici, tout en restant remplaçable par un mécanisme pub/sub plus puissant si l'échelle l'exigeait.

Critère	SSE (retenu)	WebSocket
Sens du flux	Serveur → client (unidirectionnel)	Bidirectionnel full-duplex
Protocole	HTTP/1.1 (chunked)	Protocole dédié (upgrade)
Reconnexion	Native (automatique)	À implémenter
Complexité	Faible	Élevée (au regard du besoin)
Adéquation au besoin	Élevée (notifications)	Surdimensionnée ici

Tableau 6 — Comparaison SSE vs WebSocket pour les notifications

5. Structure du projet

5.1 Organisation du monorepo et workspaces NPM

Le monorepo regroupe applications et paquets partagés sous deux espaces de travail NPM : `apps/*` (web, mobile, api) et `packages/*` (code partagé). Cette organisation offre une source de vérité unique, une gestion centralisée des dépendances et l'exécution d'une chaîne CI/CD couvrant l'ensemble. L'arborescence détaillée a été présentée en section 3.1.

5.2 Séparation des responsabilités

La séparation des responsabilités s'exprime à plusieurs niveaux : l'API Gateway concentre la sécurité transverse et le routage ; chaque micro-service porte un unique domaine métier ; le paquet `@breezy/shared` mutualise le code client (appels API, hooks, types, i18n) entre web et mobile, garantissant une cohérence stricte des contrats.

5.3 Conventions et patterns architecturaux

Le projet s'appuie sur un ensemble de patterns reconnus, qui structurent le code et facilitent la maintenance :

- **Microservices** : services indépendants, bases séparées, déploiement autonome.
- **API Gateway** : point d'entrée unique et reverse proxy centralisé.
- **Repository pattern** : GORM encapsule l'accès aux données.
- **Middleware chain** : logging, CORS, JWT et rate limiting chaînés côté Gin.
- **Token family rotation** : sécurité anti-rejeu sur les refresh tokens.
- **Server-Sent Events** : temps réel unidirectionnel simple.
- **GitOps** : infrastructure as code synchronisée depuis le dépôt via Argo CD.
- **Shared package** : réutilisation web/mobile via `@breezy/shared`.
- **Distroless containers** : images de production minimales et durcies.
- **Polyglot persistence** : une base par service, isolation totale.

6. Fonctionnalités du réseau social

Cette section détaille les fonctionnalités implémentées. Pour chacune sont précisés l'objectif, le fonctionnement technique, l'architecture associée et le bénéfice utilisateur.

6.1 Authentification complète

- **Objectif** : permettre une inscription sûre et une authentification robuste (e-mail vérifié, MFA, OAuth2).
- **Fonctionnement** : l'auth-service crée le compte en statut non vérifié, hache le mot de passe (bcrypt), envoie un e-mail de vérification (token à usage unique, expiry 24 h) ; la connexion émet un JWT (15 min) et un refresh token (30 jours) ; la MFA TOTP et l'OAuth2 (Google, GitHub) complètent le dispositif.
- **Architecture** : auth-service + auth-db ; rate limiting Valkey contre le brute-force ; e-mails via SMTP (Mailpit en dev).
- **Bénéfice** : comptes fiables, résistance aux abus et options de connexion modernes.

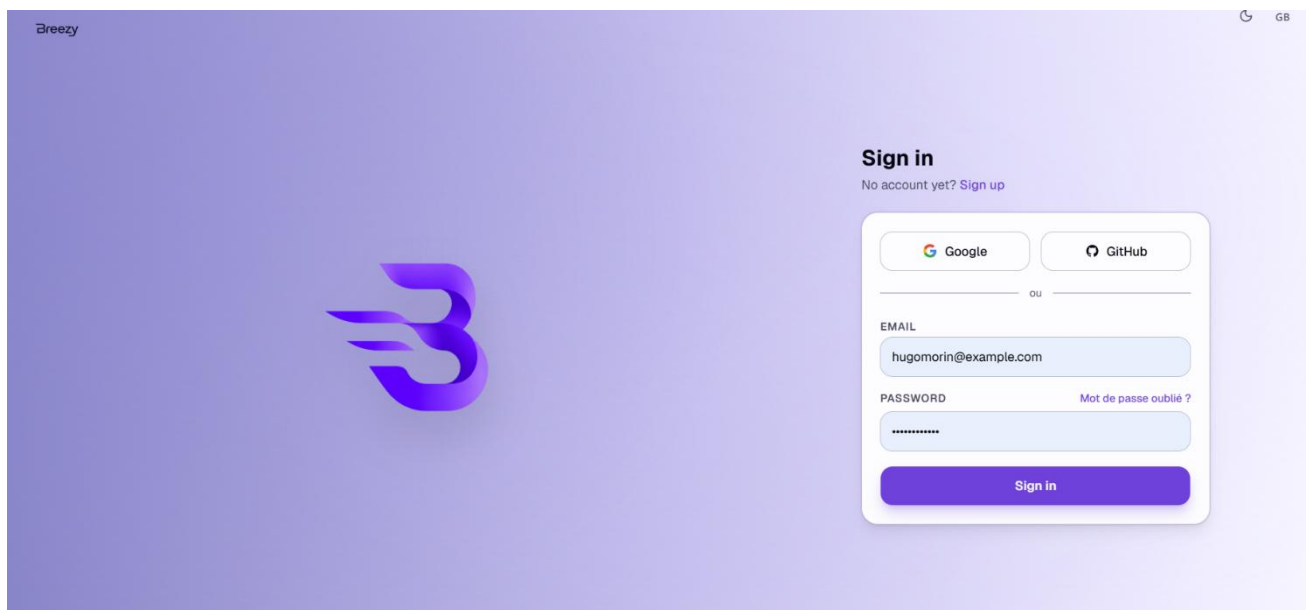


Figure 7 — Page de connexion

6.2 Profils utilisateurs

- **Objectif** : permettre à chacun de personnaliser son identité publique.
- **Fonctionnement** : nom affiché, biographie, pronoms, avatar et bannière (téléversés vers MinIO) ; paramètre public/privé.
- **Architecture** : user-service + user-db ; objets médias dans MinIO ; compteurs de followers/following.
- **Bénéfice** : une présence soignée et un contrôle de la visibilité.

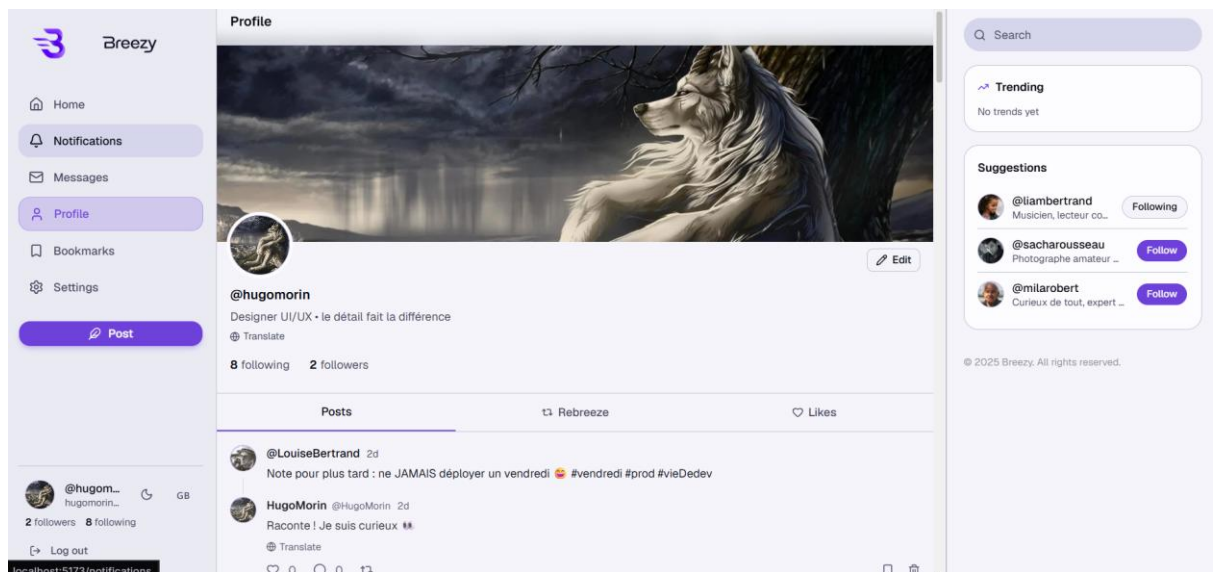


Figure 7 — Page de profil utilisateur

6.3 Publication : posts, réponses, citations, médias, sondages

- **Objectif** : publier et enrichir des contenus courts.
- **Fonctionnement** : création de posts, réponses (via parent_id), citations (quote_id), pièces jointes (image/vidéo/audio jusqu'à 300 Mo), sondages avec vote, détection automatique des hashtags et mentions.
- **Architecture** : post-service + post-db ; médias dans MinIO ; mise à jour optimiste et invalidation du cache TanStack Query côté client.
- **Bénéfice** : une composition riche et une interface réactive.

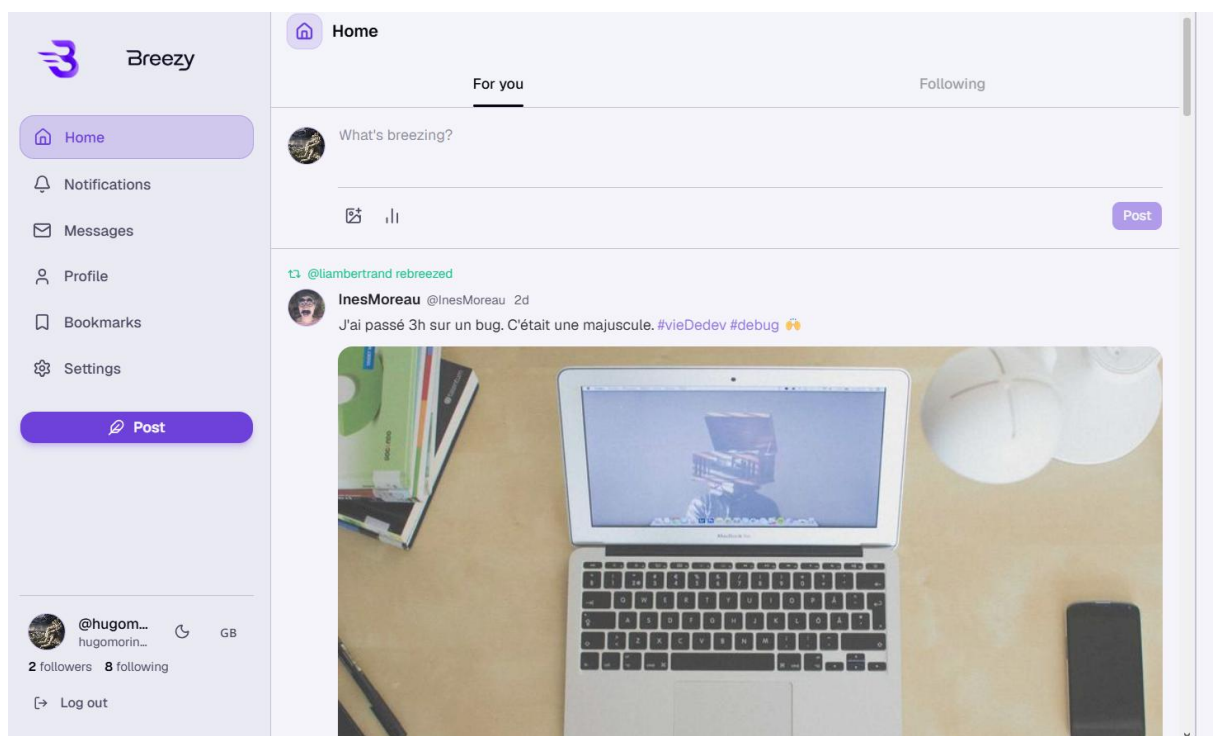


Figure 8 — Fil d'accueil et composition d'un post

6.4 Interactions : likes, rebreeze, signets

- **Objectif** : permettre des réactions légères et la curation personnelle.
- **Fonctionnement** : like, rebreeze (republication) et bookmark (signet privé), avec retour visuel immédiat.
- **Architecture** : tables likes, rebreezers, bookmarks (post-db) ; compteurs dénormalisés pour des lectures rapides.
- **Bénéfice** : engagement fluide et collections privées.

6.5 Hashtags, tendances et recherche

- **Objectif** : favoriser la découverte de contenus et de comptes.
- **Fonctionnement** : détection automatique des hashtags, feed par hashtag, hashtags en tendance, recherche d'utilisateurs et de posts.
- **Architecture** : tables hashtags / post_hashtags (post-db) ; endpoints dédiés exposés via le Gateway.
- **Bénéfice** : une exploration thématique et sociale aisée.

6.6 Suivi, comptes privés et blocage

- **Objectif** : structurer le graphe social et protéger la vie privée.
- **Fonctionnement** : follow/unfollow ; pour les comptes privés, demandes de suivi à accepter ; blocage d'utilisateurs ; le contenu privé n'est visible que des abonnés.
- **Architecture** : tables follows, follow_requests, blocks (user-db) ; le suivi d'un compte public renvoie 204, privé renvoie 202 (en attente).
- **Bénéfice** : maîtrise fine de son audience et de ses interactions.

6.7 Feed personnalisé « For You »

- **Objectif** : présenter un fil pertinent plutôt que purement chronologique.
- **Fonctionnement** : catégorisation automatique des posts par thème (theme clustering) et scoring thématique pour ordonner le feed.
- **Architecture** : post-service (champ theme / theme_inferred) ; cache Valkey pour accélérer les lectures fréquentes.
- **Bénéfice** : une découverte plus engageante.

6.8 Messagerie directe chiffrée

- **Objectif** : permettre des échanges privés 1-à-1 confidentiels.
- **Fonctionnement** : conversations, messages chiffrés au repos (AES-256-GCM), médias, suivi des non-lus (last_read_at par participant).
- **Architecture** : message-service + msg-db ; clé dérivée de MESSAGE_ENCRYPTION_KEY.
- **Bénéfice** : confidentialité des conversations même en cas d'accès à la base.

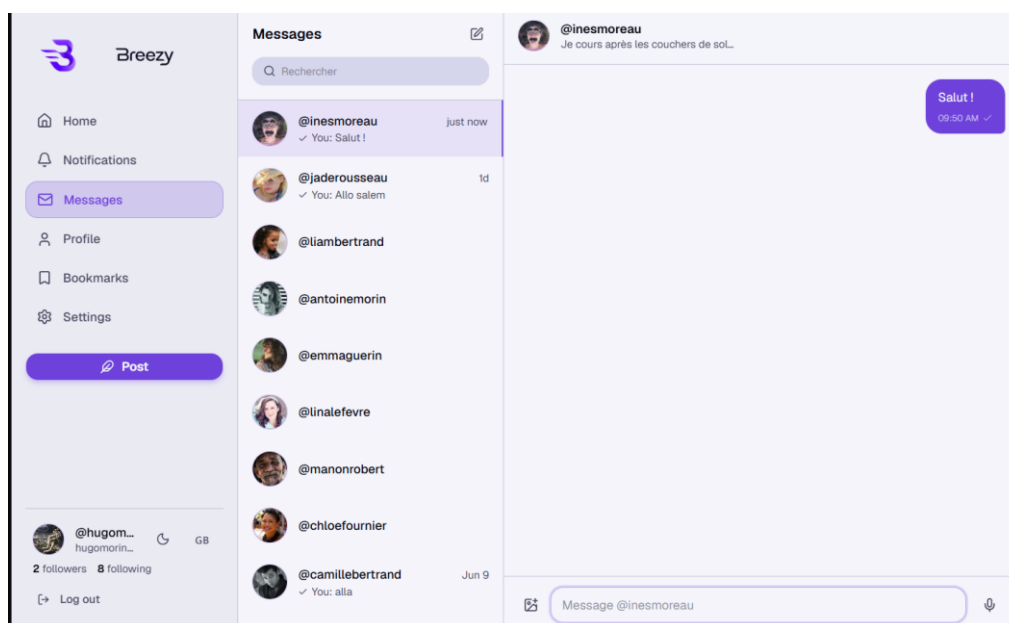


Figure 9 — Messagerie directe chiffrée

6.9 Notifications temps réel

- **Objectif** : informer l'utilisateur sans rechargement.
- **Fonctionnement** : flux SSE persistant ; notifications de likes, follows, mentions, rebreeze, messages, demandes de suivi et acceptations ; compteur de non-lus, marquage lu/non-lu.
- **Architecture** : notification-service + notif-db ; PostgreSQL LISTEN/NOTIFY comme bus interne (voir section 11).
- **Bénéfice** : une expérience vivante et immédiate.

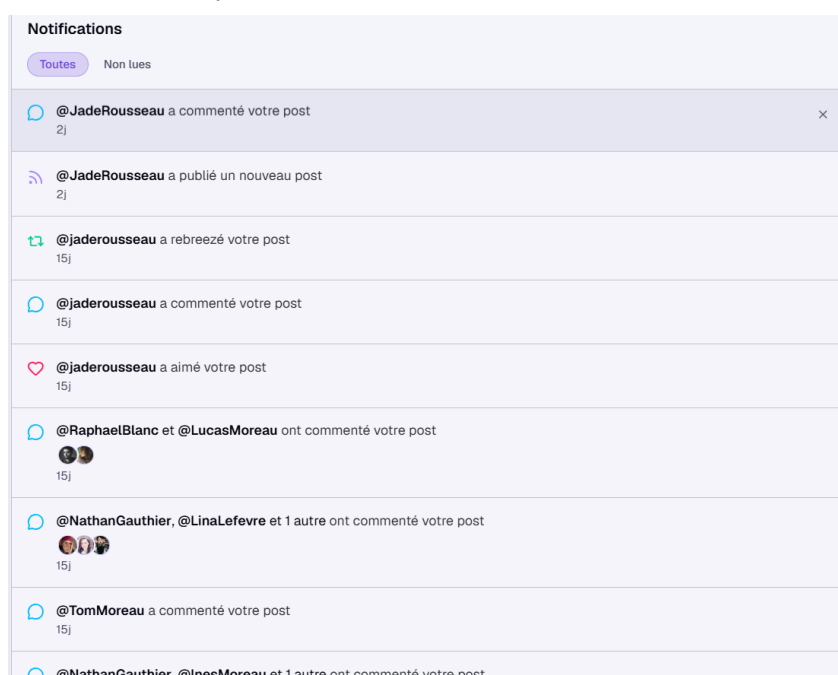


Figure 10 — Centre de notifications

6.10 Modération et administration

- **Objectif** : maintenir un espace sain.
- **Fonctionnement** : signalement de posts, historique des signalements, actions staff (résoudre, rejeter, supprimer), sanctions utilisateurs (suspension, bannissement), tableau de bord d'administration.
- **Architecture** : tables reports (post-db) et sanctions (user-db) ; rôles user/moderator/administrator.
- **Bénéfice** : des outils de gouvernance pour l'équipe et la communauté.

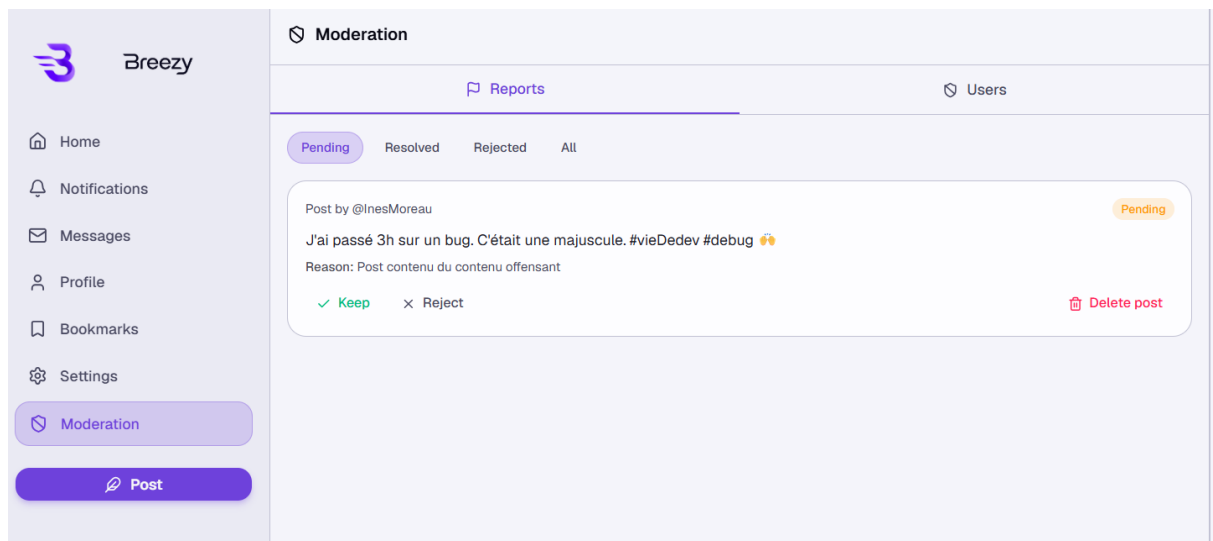


Figure 11 — Interface de modération et d'administration

7. Expérience utilisateur

7.1 Parcours et navigation

L'onboarding guide l'utilisateur de l'inscription à la première interaction, puis l'usage quotidien s'articule autour du feed, des notifications et de la messagerie. La navigation s'adapte au support : barre latérale (Sidebar) sur desktop, barre inférieure (BottomNav) sur mobile, avec un panneau de droite (RightPanel) pour les suggestions et tendances sur les grands écrans.

7.2 Design system

L'interface repose sur un design system cohérent : TailwindCSS 4 pour les utilitaires de style, les composants accessibles de shadcn/ui et Base UI, la police variable Geist et les icônes lucide-react. L'approche mobile-first garantit une expérience soignée sur petits écrans avant l'élargissement aux grands formats.

7.3 Thème, responsive et internationalisation

- **Thème clair / sombre** : géré via ThemeContext.
- **Responsive** : layout trois colonnes sur desktop, navigation compacte sur mobile, code commun au web et au natif.
- **Internationalisation** : français et anglais, détection automatique de la langue du navigateur, fichiers de traduction par fonctionnalité.

7.4 Accessibilité

L'usage de composants accessibles (shadcn/ui, Base UI) pose des fondations solides (sémantique, navigation clavier). Le renforcement de l'accessibilité contrastes, attributs ARIA, parcours clavier exhaustifs figure parmi les axes d'amélioration (section 18).

8. Frontend Web

Le frontend web (apps/breezy-web) est une SPA React 19 / Vite / TypeScript. Cette section en détaille l'architecture, les composants, la gestion d'état, les hooks, l'accès API et les optimisations.

8.1 Composants et pages

L'interface s'organise autour de composants de mise en page (AppLayout, Sidebar, BottomNav, RightPanel) et de composants métier (PostCard, PostComposer, PostActions, PollDisplay). Les pages couvrent l'ensemble des parcours : feed, profil, post, messages, notifications, recherche, signets, hashtag, abonnés/abonnements, paramètres et administration.

- **PostCard** : carte de post : auteur, contenu, médias, actions (like, rebreeze, signet, réponse).
- **PostComposer** : saisie, upload de médias, autocomplétion des mentions, création de sondages, citation de posts.
- **AppLayout** : layout trois colonnes orchestrant Sidebar, contenu et RightPanel.

8.2 Routage et protection des routes

React Router 7 distingue les routes publiques (login, register, forgot-password, verify-email, reset-password) des routes protégées nécessitant une authentification (feed, notifications, messages, profil, recherche, signets, paramètres, administration réservée au staff). La redirection s'opère selon l'état d'authentification porté par l'AuthContext.

8.3 Gestion d'état hybride

L'état est séparé en deux familles. L'état serveur (posts, utilisateurs, notifications) est géré par TanStack Query : cache, déduplication, invalidation, pagination pour l'infinite scroll, avec un staleTime par défaut de 30 secondes. L'état client (identité, thème, langue) est porté par React Context (AuthContext, ThemeContext, LanguageContext). Cette séparation évite de confondre données distantes et préférences locales.

8.4 Hooks personnalisés et accès API

Des hooks dédiés (useAuth, useFeed, usePost, useUser, useNotifications) encapsulent la logique d'accès et de cache. Le client Axios, centralisé dans @breezy/shared, injecte automatiquement le Bearer token, ajoute l'en-tête CSRF X-Requested-With et gère le rafraîchissement automatique : à la réception d'un 401, les requêtes concurrentes sont mises en file, un refresh est tenté, puis les requêtes sont rejouées sans condition de course.

8.5 Gestion de l'upload de médias

L'upload accepte images, vidéos et audio jusqu'à 300 Mo, avec conversion HEIC→JPEG sur iOS (heic2any), suivi de progression et timeout adapté aux fichiers volumineux. Les fichiers transitent vers /media/upload puis MinIO.



8.6 Optimisation des performances et configuration Vite

Le frontend tire parti du React Compiler (mémoïsation automatique des composants), du cache de TanStack Query (moins d'appels redondants), de la pagination et du lazy loading. Vite assure un build optimisé et un proxy /api/* vers l'API Gateway en développement, avec des modes de build distincts pour le web et le mobile.

8.7 Paquet partagé @breezy/shared

Ce paquet centralise le code commun au web et au mobile : clients API (auth, posts, users, messages, notifications, media...), contexte (auth, thème, langue), hooks, types TypeScript et internationalisation. Il garantit que les deux plateformes consomment exactement les mêmes contrats et la même logique.

9. Backend Micro-services Go

9.1 API Gateway

L'API Gateway est la façade unique du backend. Il valide les JWT, applique la limitation de débit, vérifie la révocation de session via Valkey, impose CORS et CSRF, puis relaie chaque requête au service approprié. En centralisant ces préoccupations, il allège les services internes, qui se concentrent sur leur domaine.

9.2 Services et responsabilités

Service (port)	Responsabilités	Exemples d'endpoints
auth (3104)	Inscription, login/refresh/logout, vérification e-mail, reset, MFA, OAuth2	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/mfa/verify
user (3102)	Profils, follow/blocage, demandes de suivi, suggestions, sanctions	GET /users/:id, POST /users/:id/follow, PATCH /users/me
post (3103)	Posts, likes, rebreeze, signets, médias, hashtags, sondages, feed, signalements	POST /posts, GET /posts/for-you, POST /posts/:id/likes
message (3105)	Conversations et messages chiffrés	POST /conversations, GET /conversations/:id/messages
notification (3106)	Notifications + flux SSE	GET /notifications, GET /subscriptions (SSE)
core (3101)	Service d'extension (base minimale)	—

Tableau 7 — Micro-services : responsabilités et endpoints représentatifs

Les listes d'endpoints ci-dessus sont volontairement représentatives et non exhaustives, afin de préserver la lisibilité ; la documentation Swagger fait foi pour le contrat complet.

9.3 Communication inter-services

Les services communiquent en HTTP interne, authentifié par une clé d'API partagée (INTERNAL_API_KEY), selon un modèle fire-and-forget pour les traitements asynchrones (notamment l'émission de notifications). L'absence délibérée de broker (Kafka/RabbitMQ) maintient la simplicité opérationnelle, au prix d'un couplage temporel accepté à cette échelle.

9.4 Middlewares, validation et rôles

La chaîne de middlewares Gin applique successivement journalisation, CORS, validation JWT et limitation de débit. La validation des entrées (par exemple la longueur des messages) et la gestion uniforme des erreurs sécurisent les services. Trois rôles structurent les permissions : user, moderator et administrator, ce dernier accédant aux fonctions de modération et d'administration.

10. Base de données

10.1 Architecture polyglotte

Chaque micro-service possède sa base PostgreSQL dédiée, ce qui assure une isolation totale des données et un couplage minimal. Les références inter-services sont des identifiants logiques sans contrainte de clé étrangère physique ; la cohérence relève de la logique applicative. Ce choix favorise l'autonomie d'évolution et le déploiement indépendant des services.

10.2 Modèles structurants

Base	Tables principales (extrait)
auth-db	auth_users, auth_refresh_tokens (family_id), auth_mfa, auth_email_verifications, auth_password_resets, auth_social_accounts
user-db	users, follows, follow_requests, blocks, sanctions
post-db	posts (parent_id, quote_id, theme), likes, rebreezers, bookmarks, media, hashtags, post_hashtags, polls, poll_options, poll_votes, reports
msg-db	conversations, participants (last_read_at), messages (contenu chiffré)
notif-db	notifications, subscriptions

Tableau 8 — Tables principales par base (extrait représentatif)

10.3 Relations et schéma

À l'intérieur d'une base, les relations sont classiques (clés étrangères, par exemple poll_options référence polls, ou posts.parent_id référence posts pour les réponses). Entre bases, les liens sont logiques.

10.4 Choix de PostgreSQL

PostgreSQL apporte des garanties transactionnelles ACID, une grande robustesse et un mécanisme LISTEN/NOTIFY réutilisé comme bus d'événements interne pour le temps réel. Ces atouts, combinés à la maturité de l'écosystème, en font un socle naturel pour l'ensemble des services.

11. Temps réel

Breezy diffuse les notifications en direct grâce aux Server-Sent Events, adossés au mécanisme LISTEN/NOTIFY de PostgreSQL.

11.1 Flux de notification de bout en bout

- **1) Abonnement** : le client ouvre une connexion SSE persistante sur GET /subscriptions.
- **2) Événement** : une action survient (like, follow, message...).
- **3) Notification interne** : le service concerné appelle l'endpoint interne de notification.
- **4) Persistance** : le notification-service enregistre la notification en base.
- **5) Signal** : PostgreSQL LISTEN/NOTIFY déclenche un événement.
- **6) Diffusion** : le gestionnaire SSE émet (flush) l'événement vers le client.
- **7) Mise à jour UI** : le client React reçoit l'événement et met à jour l'interface.

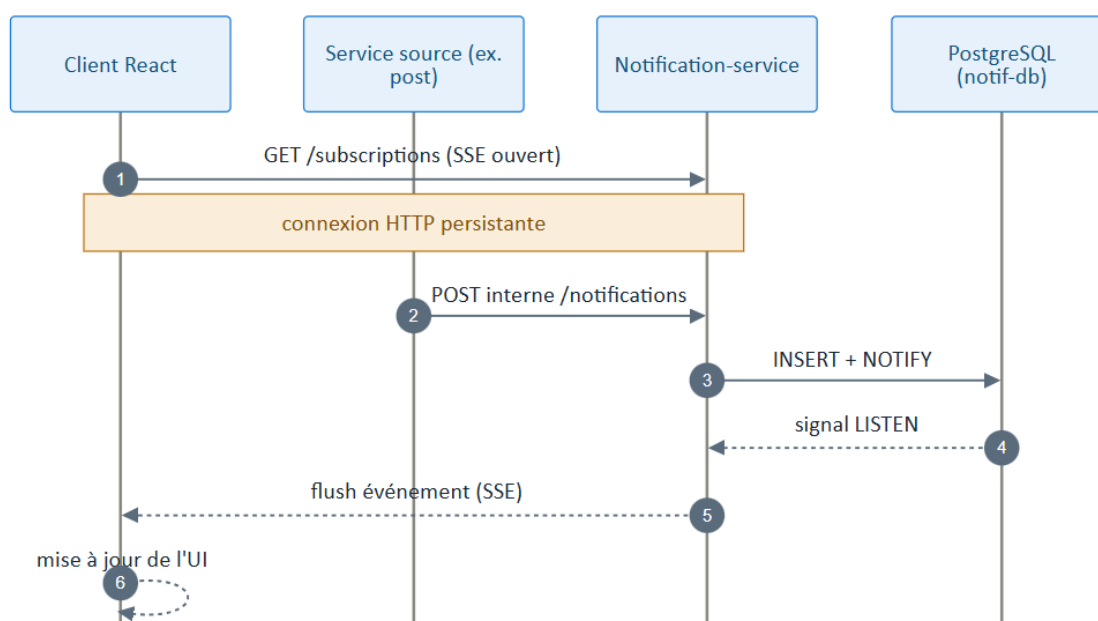


Figure 12 — Flux temps réel (SSE + LISTEN/NOTIFY)

11.2 Types d'événements et justification du choix

Les événements couvrent likes, nouveaux abonnés, mentions, rebreeze, demandes de suivi, acceptations et nouveaux messages. Le choix de SSE plutôt que WebSocket (cf. section 4.4) tient à la nature unidirectionnelle du besoin : SSE offre simplicité, reconnexion native et compatibilité HTTP standard. À plus grande échelle, le bus interne pourrait être remplacé par un pub/sub dédié (Redis, par exemple).

12. Sécurité

La sécurité est traitée en profondeur, du chiffrement des mots de passe au durcissement des conteneurs.

12.1 Authentification JWT et refresh tokens

Les jetons d'accès JWT (HS256 ou RS256, configurable) sont de courte durée (15 minutes) et accompagnés d'un refresh token longue durée (30 jours). Chaque rafraîchissement génère un nouveau couple de jetons (rotation). Une « famille » (family_id) trace la lignée des refresh tokens afin de détecter les rejeux ; l'adresse IP et le User-Agent sont conservés pour repérer les anomalies, et les sessions sont révocables via Valkey. La rotation de clé (clé précédente conservée) permet une transition sans interruption.

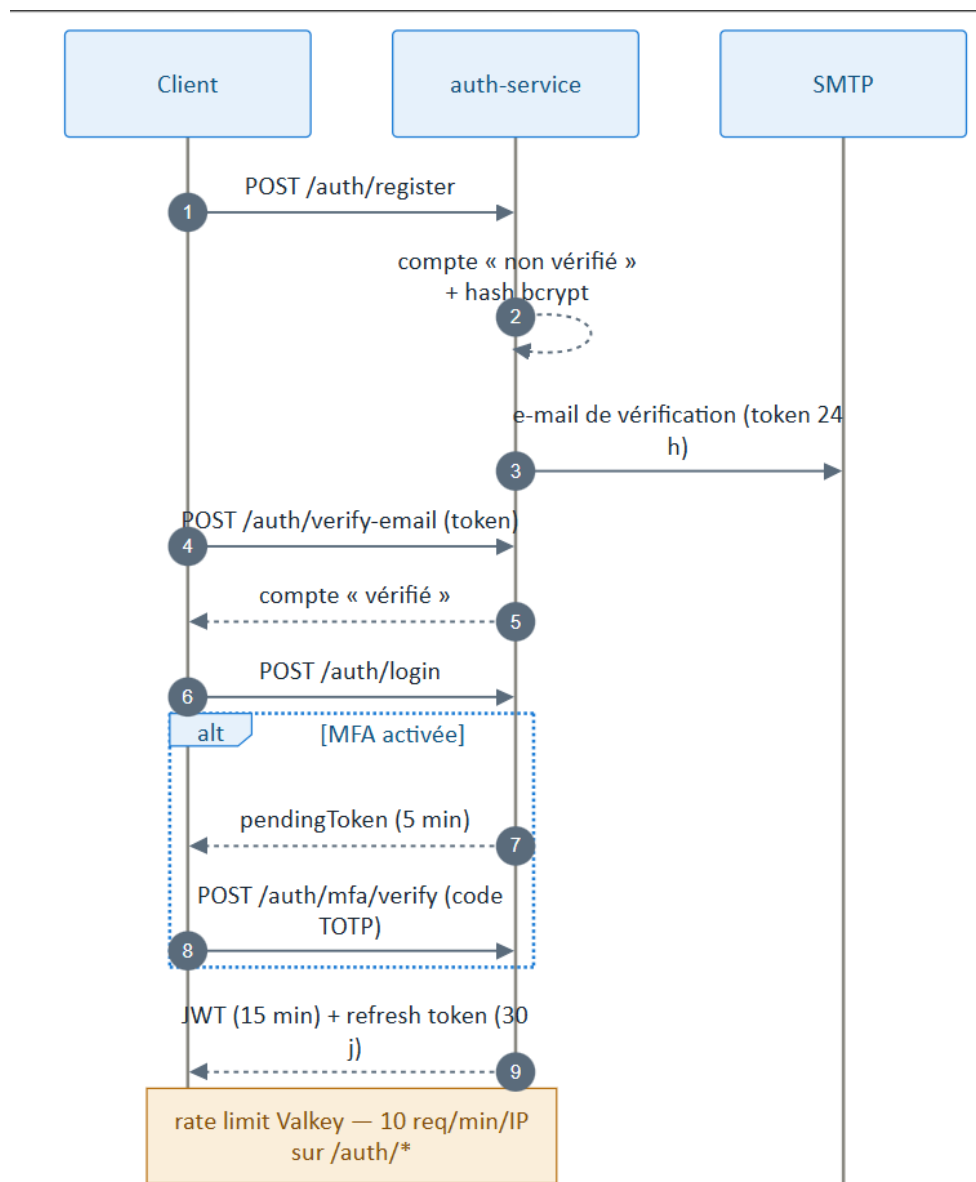


Figure 13 — Flux d'inscription, vérification d'e-mail et connexion

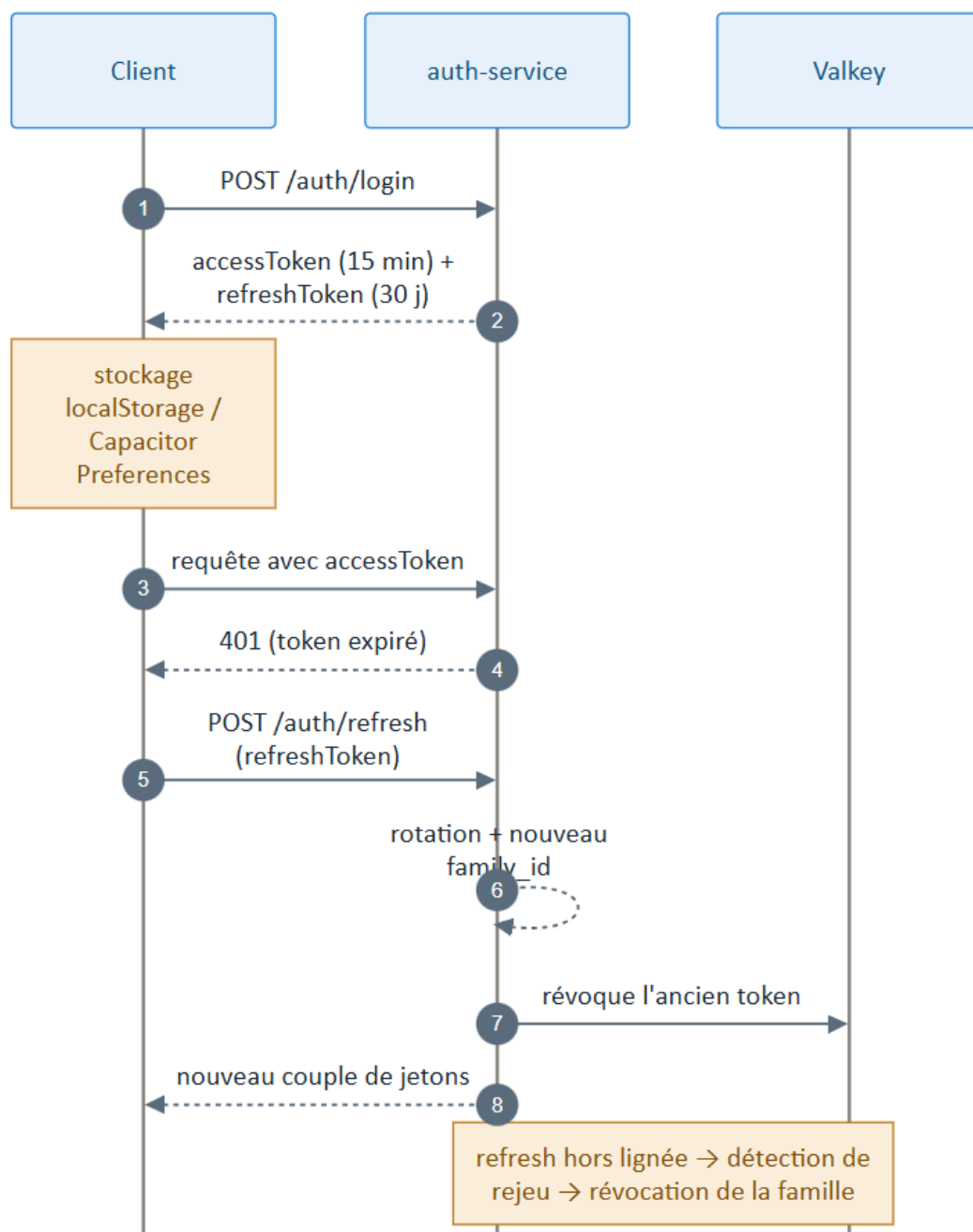


Figure 14 — Rotation sécurisée du refresh token

12.2 MFA, OAuth2 et vérifications

- **MFA (TOTP)** : secret généré, QR code, confirmation par code, 10 codes de secours hachés ; au login, échange d'un pendingToken (5 min) contre les jetons réels.
- **OAuth2** : fournisseurs Google et GitHub ; flux authorization_code → échange court → JWT ; un utilisateur peut lier plusieurs comptes.



- **Vérification d'e-mail** : token SHA-256 à usage unique (24 h) ; connexion impossible avant vérification ; réponse 202 même si l'e-mail est inconnu (anti-énumération).
- **Réinitialisation de mot de passe** : token à usage unique (60 min) ; toutes les sessions révoquées à la réinitialisation.

12.3 Chiffrement, limitation de débit et durcissement

- Chiffrement des messages : AES-256-GCM au repos, clé dérivée de MESSAGE_ENCRYPTION_KEY.
- **Rate limiting** : Valkey : 10 req/min/IP sur l'authentification, 120 req/min/utilisateur sur l'API.
- **CORS / CSRF** : liste blanche d'origines ; en-tête X-Requested-With obligatoire.
- **Mots de passe** : hachage bcrypt (golang.org/x/crypto).
- **Conteneurs durcis** : images distroless : sans shell, non-root, no-new-privileges, capacités retirées, système de fichiers en lecture seule.
- **Secrets** : Sealed Secrets en production (secrets chiffrés versionnés).

13. Performances

La réactivité est traitée des deux côtés de la pile.

- **React Compiler** : mémoïsation automatique des composants, réduisant les rendus superflus.
- **TanStack Query** : cache, déduplication et staleTime limitent les appels redondants.
- **Pagination & infinite scroll** : chargement progressif des listes (feed, profils).
- **Lazy loading** : chargement différé des composants lourds.
- **Cache Valkey** : accélère côté backend les lectures fréquentes (feed, sessions).
- **MinIO** : sert les médias de manière scalable, compatible CDN.
- **Images multi-stage** : binaires Go légers et images distroless réduites.
- **Build Vite** : bundling optimisé et démarrage rapide.

La conception sans état des services (JWT auto-porté) autorise un scaling horizontal natif : on multiplie les instances sans coordination de session, Valkey assurant la limitation de débit distribuée et la révocation.

14. Déploiement et infrastructure

14.1 Environnements et développement local

Deux environnements sont distingués (dev et production). En local, Docker Compose orchestre l'ensemble : six instances PostgreSQL, les sept services Go (distroless), MinIO, Valkey, ainsi que des outils de développement (Mailpit pour capturer les e-mails, pgAdmin pour les bases, Swagger UI pour la documentation). Des réseaux Docker séparés isolent l'accès externe, la communication inter-services et l'accès au cache.

14.2 Pipeline CI/CD (GitHub Actions)

Le pipeline se déclenche sur chaque push et sur les pull requests vers main et develop. Il enchaîne lint, tests, build, publication des images et mise à jour des manifests Kubernetes.

- **Lint** : golangci-lint (Go) et ESLint (TypeScript/React).
- **Test** : tests unitaires Go, tests d'intégration (testcontainers-go + PostgreSQL réel), Vitest (front) ; rapports de couverture en artefacts.
- **Build** : compilation des binaires.
- **Publish** : images Docker multi-plateforme publiées sur GHCR, taguées par SHA git et tag flottant (latest/dev), avec cache par couche.
- **Update K8s** : mise à jour des tags d'images dans les overlays Kustomize : push direct sur main, création de PR sur develop (branche protégée).

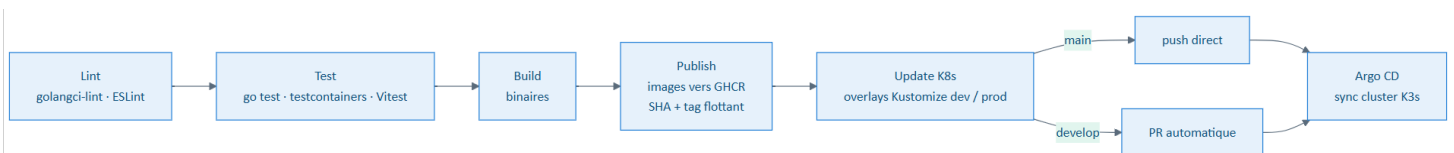


Figure 15 — Pipeline CI/CD et mise à jour GitOps

14.3 Kubernetes, GitOps et TLS

Le cluster repose sur K3s (Kubernetes léger, Traefik intégré). Argo CD assure le GitOps selon un modèle app-of-apps, avec des applications distinctes pour dev (branche develop) et prod (branche main). Kustomize gère les overlays sans duplication. Cert-Manager et Let's Encrypt automatisent les certificats TLS (challenge DNS-01 via Route53), External DNS gère les enregistrements, et Sealed Secrets permet de versionner des secrets chiffrés.



14.4 Stratégie d'images et monitoring

Les images Docker s'appuient sur des builds multi-stage (étape de compilation, étape de développement avec hot-reload, étape d'exécution distroless). Les contraintes de production (limites CPU/mémoire par service, système de fichiers en lecture seule, capacités retirées) renforcent la sécurité et la prévisibilité. Le monitoring s'appuie sur les mécanismes natifs de Kubernetes ; l'observabilité avancée figure parmi les évolutions envisagées.

15. Gestion de projet

15.1 Méthodologie GitHub-centric

Le projet a été piloté avec GitHub Projects comme outil central de suivi, combiné aux fonctionnalités natives de GitHub : Issues, Pull Requests et règles de protection de branches. Le développement suit une approche itérative et incrémentale, chaque incrément livrant des fonctionnalités démontrables.

- **GitHub Projects** : tableau de suivi (backlog/kanban) pour visualiser l'avancement, assigner et prioriser.
- **Issues** : chaque fonctionnalité, bug ou amélioration tracé comme unité de travail (spécification, discussion, référence).
- **Pull Requests obligatoires** : aucun code mergé directement sur develop ou main ; tout passe par une PR.
- **Revue de code obligatoire** : chaque PR relue et approuvée par au moins un autre membre.
- **CI bloquante** : lint, tests et build doivent être au vert avant tout merge (double validation : humaine + automatisée).
- **Lien Issues ↔ PRs** : fermeture automatique des issues au merge, traçabilité complète.

15.2 Stratégie de branches et bénéfices

La gestion de version repose sur Git avec un modèle par branches dédiées : chaque évolution est développée sur une branche préfixée feature/ (nouvelle fonctionnalité) ou fix/ (correction), créée depuis la branche principale, puis intégrée via une pull request soumise à la revue et à la CI avant fusion. Les branches develop et main sont protégées : la mise à jour des manifests Kubernetes sur develop passe elle-même par des PR créées automatiquement par le pipeline. Cette organisation maintient la qualité du code, la responsabilité collective et un historique décisionnel tracé dans les PR.

15.3 Documentation et répartition

La documentation est générée à partir du code : Swagger décrit les endpoints de l'API (contrats, schémas, codes de retour). Cette approche garde la documentation alignée avec l'implémentation.

Répartition des rôles

Membre / Rôle principal / Responsabilités : Antonin RABATEL, Développeur Back-end, Responsable Docker, Tests, User ; Mathéo CARVAL, Développeur back-end, Responsable Posts, Notifications, Mobile ; Youcef AFANE, Développeur Front-end, Responsable UX ; Enzo CADIERE, Administrateur Système et Réseau, Responsable Kubernetes.

Charge estimée : 60 jours-homme (jours de perte avec prosits/workshops) ; jalons : Projet découpé en 4 semaines donc 4 jalons. Jalon 1 : Fondations et Infrastructures ; Jalon 2 : Fonctionnalités sociales coeur ; Jalon 3 : Sécurité avancée et expérience utilisateur ; Jalon 4 : Tests, Mobile et finalisation



Vue d'avancement — Projet DADdy (Breezy)

Snapshot du GitHub Project · 89 tickets

Avancement **92 %** Terminés **82** En cours **3** Backlog **4**

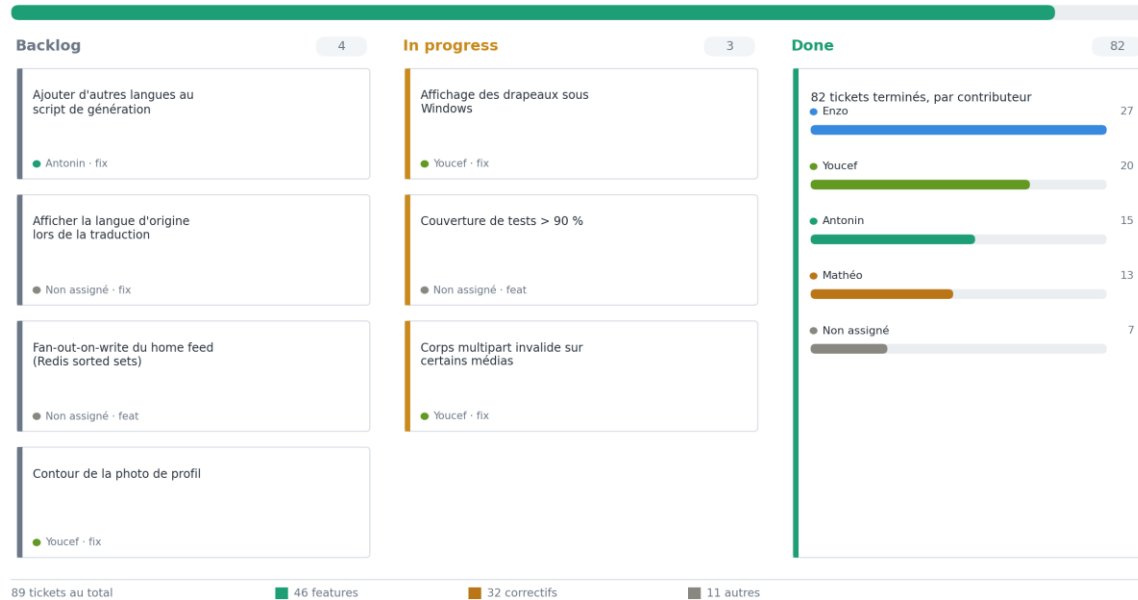


Figure 16 — Vue d'avancement du projet

16. Application mobile

L'application mobile étend Breezy aux plateformes iOS et Android tout en réutilisant l'intégralité du code web.

16.1 Architecture Capacitor et partage de code

Capacitor 6 encapsule le bundle web (compilé en mode mobile) dans une WebView native ; le pont expose les capacités du téléphone sous forme d'API JavaScript. Le code provient de @breezy/shared : la logique, les écrans et les contrats sont identiques au web, ce qui élimine la divergence fonctionnelle entre plateformes.

16.2 Spécificités natives et builds

- **Capacités natives** : caméra, notifications push, partage, haptique, réseau, barre de statut, splash screen.
- **Stockage des jetons** : Capacitor Preferences (chiffré nativement) en remplacement du localStorage.
- **Builds** : applications iOS et Android générées à partir du même codebase via le mode build mobile de Vite.

Fonctionnellement, l'application mobile est équivalente au web ; ses différences relèvent de l'intégration native (stockage chiffré, accès matériel) et de l'isolation de la WebView.

17. Difficultés techniques rencontrées

Le projet a confronté l'équipe à plusieurs défis significatifs, inhérents à une architecture distribuée complète. Les principaux sont présentés ci-dessous avec la manière dont ils ont été abordés.

- **Cohérence inter-services sans FK** : l'absence de clés étrangères physiques entre bases a imposé de gérer les références (ex. `user_id` dans `post-db`) comme identifiants logiques, la cohérence étant assurée par la logique applicative et des vérifications côté service.
- **Rotation des refresh tokens** : la mise en œuvre des familles de jetons (`family_id`), de la détection de rejeu et de la révocation a demandé une attention particulière pour éviter de déconnecter les sessions légitimes.
- **Latence des appels inter-services** : certains parcours (ex. vérification de confidentialité impliquant `gateway` → `user` → `post`) ont nécessité du cache (Valkey) pour contenir la latence cumulée.
- **Upload de fichiers volumineux** : la gestion d'uploads jusqu'à 300 Mo a imposé un suivi de progression, des timeouts adaptés et la conversion HEIC→JPEG sur iOS.
- **Montée en charge des SSE** : l'usage de PostgreSQL LISTEN/NOTIFY comme bus interne a été éprouvé, en gardant à l'esprit qu'un pub/sub dédié serait préférable à très grande échelle.
- **Synchronisation du schéma via GORM** : la cohérence des migrations entre services et environnements a requis de la rigueur, notamment vers la production.
- **Build mobile Capacitor** : l'alignement des dépendances web avec les builds iOS/Android et le mode mobile de Vite a demandé des ajustements.
- **Orchestration de six bases PostgreSQL** : exploiter six instances indépendantes (StatefulSets) a complexifié la configuration et le suivi.
- **Chiffrement des messages** : concilier le chiffrement au repos (AES-256-GCM) avec les contraintes de lecture a structuré le modèle du message-service.

18. Évolutions futures envisageables

18.1 Fonctionnelles

- **Stories éphémères** : contenus temporaires.
- **Audio/vidéo en messagerie** : via WebRTC.
- **Recommandations avancées** : apprentissage automatique sur les thèmes.
- **Modération assistée par IA** : classification des signalements.
- **API publique** : système OAuth pour développeurs tiers.
- **Monétisation** : compte premium, boost de publication.

18.2 Techniques

- **Bus d'événements dédié** : Redis Streams ou Kafka pour remplacer LISTEN/NOTIFY à grande échelle.
- **CDN devant MinIO** : diffusion accélérée des médias.
- **Observabilité avancée** : OpenTelemetry, Grafana, Loki.
- **Partitionnement / sharding** : mise à l'échelle des bases PostgreSQL.
- **Accessibilité et tests end-to-end** : sécuriser les parcours critiques et renforcer l'inclusivité.

19. Conclusion

Le projet Breezy a permis de concevoir et de réaliser un réseau social complet, web et mobile, en mobilisant une architecture distribuée moderne : sept micro-services Go conteneurisés, une persistance polyglotte PostgreSQL, un cache et une limitation de débit Valkey, un stockage objet MinIO, des notifications temps réel par SSE et une sécurité de premier plan (JWT à rotation, MFA, OAuth2, chiffrement des messages, conteneurs durcis).

La cohérence de l'ensemble monorepo, paquet partagé web/mobile, GitOps via Argo CD, CI/CD bloquante et tests d'intégration sur bases réelles témoigne d'une réelle maturité d'ingénierie. Le projet démontre qu'une équipe réduite peut maîtriser une chaîne complète, de la conception au déploiement automatisé.

Sur le plan pédagogique, Breezy a consolidé des compétences essentielles : architecture distribuée, sécurité applicative, industrialisation et collaboration outillée. Les évolutions identifiées ouvrent des perspectives concrètes vers un service plus riche, plus observable et plus largement déployable.

Le projet nous a permis de mener à bien, en équipe, la réalisation d'un réseau social distribué complet, de la conception à la mise en œuvre. La répartition du travail s'est articulée autour des compétences et des appétences de chacun : Antonin a pris en charge la stack Docker, le user-service et les tests ; Mathéo a développé le notification-service, le posts-service et l'application mobile ; Youcef s'est concentré sur le front-end, l'UX et l'algorithme du fil d'actualité ; Enzo a réalisé l'auth-service et l'orchestration Kubernetes. Cette organisation, à la fois spécialisée et complémentaire, a permis de faire avancer les différents services en parallèle.

La principale difficulté rencontrée a été la prise en main de Go par la majorité de l'équipe. Cet investissement s'est toutefois révélé pleinement justifié : les performances et le niveau de sécurité offerts par le langage répondent précisément aux exigences d'une architecture distribuée comme la nôtre.

Sur le plan des enseignements, ce projet a surtout consolidé notre capacité à développer en équipe sur une base de code commune, ainsi que notre maîtrise de l'intégration continue via GitHub Actions, désormais ancrée dans nos réflexes de travail.

Pour un prochain projet, nous identifions plusieurs axes de progression : affiner la planification en amont, intégrer les tests plus tôt dans le cycle de développement plutôt qu'en aval, et instaurer des réunions de synchronisation plus régulières afin que chacun reste informé de l'avancement des autres.



Table des figures

Figure 1 — Architecture globale de l'écosystème Breezy	9
Figure 2 — Architecture des microservices et flux de communication	11
Figure 3 — Architecture en couches du frontend	12
Figure 4 — Architecture mobile Capacitor (pont natif)	13
Figure 5 — Persistance polyglotte (une base par service)	14
Figure 6 — Stockage et service des médias (MinIO)	15
Figure 7 — Page de profil utilisateur	20
Figure 8 — Fil d'accueil et composition d'un post	20
Figure 9 — Messagerie directe chiffrée	22
Figure 10 — Centre de notifications	23
Figure 11 — Interface de modération et d'administration	24
Figure 12 — Flux temps réel (SSE + LISTEN/NOTIFY)	30
Figure 13 — Flux d'inscription, vérification d'e-mail et connexion	31
Figure 14 — Rotation sécurisée du refresh token	32
Figure 15 — Pipeline CI/CD et mise à jour GitOps	35

Glossaire

Ce glossaire regroupe les principaux termes techniques, technologies et concepts employés dans ce rapport, dans le contexte du projet Breezy.

Terme	Définition
ACID	Propriétés (Atomicité, Cohérence, Isolation, Durabilité) garantissant la fiabilité des transactions d'une base relationnelle comme PostgreSQL.
API	Interface de programmation exposant des points d'accès (endpoints) permettant à des programmes de communiquer entre eux.
API Gateway	Point d'entrée unique du backend qui route les requêtes vers les micro-services et centralise authentification, CORS et limitation de débit.
Argo CD	Outil de déploiement continu en GitOps synchronisant l'état du cluster Kubernetes avec les manifests versionnés dans Git.
Backend	Partie serveur de l'application (micro-services Go) en charge de la logique métier, de l'accès aux données et des API.
Base UI / shadcn/ui	Bibliothèques de composants d'interface accessibles et personnalisables utilisées côté frontend.
@breezy/shared	Paquet partagé du monorepo regroupant les types et utilitaires communs au web et au mobile.
Capacitor	Pont (runtime) empaquetant l'application web React en application mobile native iOS/Android en réutilisant le même code.
Cert-Manager	Composant Kubernetes automatisant l'émission et le renouvellement des certificats TLS via Let's Encrypt.
CI/CD	Intégration et déploiement continus : chaîne automatisée de lint, tests, build et publication des images (ici via GitHub Actions).
CORS	Mécanisme de sécurité du navigateur contrôlant les requêtes inter-origines, géré au niveau de la gateway.
Distroless	Image Docker minimale, sans shell ni gestionnaire de paquets, réduisant la surface d'attaque.
Docker	Technologie de conteneurisation packageant une application et ses dépendances en images reproductibles.
Endpoint	Point d'accès d'une API identifié par une URL et une méthode HTTP.

Feed « For You »	Fil d'actualité personnalisé proposant à chaque utilisateur des publications sélectionnées selon ses centres d'intérêt.
Frontend	Partie cliente de l'application (SPA React) avec laquelle l'utilisateur interagit dans le navigateur.
GHCR	GitHub Container Registry : registre d'images de conteneurs hébergé par GitHub où sont publiées les images du projet.
Gin	Framework HTTP léger et rapide pour Go, utilisé pour le routage et la chaîne de middlewares.
GitOps	Pratique pilotant le déploiement de l'infrastructure de façon déclarative depuis un dépôt Git pris comme source de vérité.
Go (Golang)	Langage compilé et typé statiquement, doté de la concurrence par goroutines, employé pour les micro-services backend.
GORM	ORM pour Go facilitant l'accès aux données, les migrations et le repository pattern.
Goroutine	Unité d'exécution concurrente légère propre au langage Go.
Hashtag	Mot-clé précédé du caractère « # » permettant d'indexer et de regrouper les publications par thème.
HMR	Hot Module Replacement : rechargement à chaud des modules pendant le développement (offert par Vite), sans recharger toute la page.
Hook (React)	Fonction React gérant l'état et le cycle de vie des composants ; le projet définit des hooks personnalisés d'accès à l'API.
i18n	Internationalisation : conception permettant d'adapter l'interface à plusieurs langues et formats régionaux.
JWT	JSON Web Token : jeton signé contenant les informations d'authentification, vérifié à chaque requête protégée.
K3s	Distribution Kubernetes légère intégrant Traefik, utilisée pour l'orchestration des conteneurs.
Kubernetes	Système d'orchestration de conteneurs gérant le déploiement, le scaling et la résilience des services.
Kustomize	Outil de configuration Kubernetes par overlays (dev/prod) évitant la duplication de manifests.
Let's Encrypt	Autorité de certification délivrant gratuitement des certificats TLS automatisés.
Like	Interaction permettant d'indiquer son appréciation d'une publication.

LISTEN/NOTIFY	Bus d'événements natif de PostgreSQL utilisé pour diffuser les notifications en temps réel.
MFA	Multi-Factor Authentication : authentification multifacteur ajoutant une étape de vérification au-delà du mot de passe.
Microservices	Architecture découpant l'application en services autonomes, déployables et évolutifs indépendamment.
Middleware	Fonction intercalée dans le traitement d'une requête HTTP (authentification, CORS, logs, rate limit).
MinIO	Solution de stockage objet auto-hébergée, compatible avec l'API Amazon S3, utilisée pour les médias.
Monorepo	Dépôt unique regroupant l'ensemble des projets (frontend, backend, paquet partagé), gérés via des workspaces NPM.
NPM Workspaces	Fonctionnalité de NPM permettant de gérer plusieurs paquets au sein d'un même dépôt.
OAuth2	Protocole d'autorisation déléguée permettant la connexion via un fournisseur tiers.
ORM	Object-Relational Mapping : technique reliant les objets du code aux tables d'une base relationnelle (ici GORM).
Persistence polyglotte	Approche où chaque micro-service possède sa propre base de données dédiée, assurant isolation et couplage minimal.
Post	Publication courte (texte et/ou médias) au cœur du réseau social.
PostgreSQL	Système de gestion de base de données relationnelle (ACID) ; chaque service dispose de sa base dédiée.
Rate limiting	Limitation de débit : restriction du nombre de requêtes par utilisateur pour protéger les services des abus.
React	Bibliothèque JavaScript de construction d'interfaces utilisateur par composants.
React Router	Bibliothèque de routage déclaratif côté client pour React, gérant aussi la protection des routes.
Rebreeze	Repartage d'une publication d'un autre utilisateur (équivalent du « retweet »).
Refresh token	Jeton de longue durée permettant de renouveler un JWT expiré sans nouvelle connexion.
Repository pattern	Patron d'architecture isolant l'accès aux données derrière une interface dédiée.

Responsive design	Conception d'interface s'adaptant automatiquement à la taille de l'écran (web et mobile).
Route53	Service DNS d'AWS utilisé pour le challenge DNS-01 de validation des certificats TLS.
S3	API de stockage objet d'Amazon, standard auquel MinIO est compatible.
Sealed Secrets	Mécanisme chiffrant les secrets afin de les versionner en toute sécurité dans Git.
Signet	Bookmark : enregistrement d'une publication pour la retrouver ultérieurement.
Sondage	Type de publication proposant un vote entre plusieurs options.
SPA	Single Page Application : application web chargée une seule fois et mise à jour dynamiquement sans rechargement de page.
SSE	Server-Sent Events : flux unidirectionnel serveur → client utilisé pour les notifications temps réel.
TailwindCSS	Framework CSS « utility-first » assurant la cohérence du design system et un bundle optimisé.
TanStack Query	Bibliothèque de gestion de l'état serveur (cache, déduplication, invalidation, pagination) côté frontend.
Tendances	Classement des hashtags et sujets les plus populaires du moment.
testcontainers-go	Bibliothèque lançant des bases PostgreSQL éphémères réelles pour des tests d'intégration fidèles.
TLS	Transport Layer Security : protocole de chiffrement sécurisant les communications réseau (HTTPS).
Traefik	Reverse proxy / ingress controller intégré à K3s assurant le routage entrant.
TypeScript	Sur-ensemble typé de JavaScript apportant le typage statique au frontend.
Valkey	Magasin clé-valeur en mémoire compatible Redis, utilisé pour le cache et la limitation de débit.
Vite	Outil de build moderne offrant un démarrage et un rechargement à chaud (HMR) très rapides, en modes web et mobile.
WebSocket	Protocole de communication bidirectionnel temps réel ; écarté ici au profit des SSE.